

to do with each other. Yes, Monad factors out code duplication among them, but what *is* a monad exactly? What does “monad” *mean*?

You may be used to thinking of interfaces as providing a relatively complete API for an abstract data type, merely abstracting over the specific representation. After all, a singly linked list and an array-based list may be implemented differently behind the scenes, but they’ll share a common interface in terms of which a lot of useful and concrete application code can be written. Monad, like Monoid, is a more abstract, purely algebraic interface. The Monad combinators are often just a small fragment of the full API for a given data type that happens to be a monad. So Monad doesn’t generalize one type or another; rather, many vastly different data types can satisfy the Monad interface and laws.

We’ve seen three minimal sets of primitive Monad combinators, and instances of Monad will have to provide implementations of one of these sets:

- unit and flatMap
- unit and compose
- unit, map, and join

And we know that there are two monad laws to be satisfied, associativity and identity, that can be formulated in various ways. So we can state plainly what a monad *is*:

A monad is an implementation of one of the minimal sets of monadic combinators, satisfying the laws of associativity and identity.

That’s a perfectly respectable, precise, and terse definition. And if we’re being precise, this is the *only* correct definition. A monad is precisely defined by its operations and laws; no more, no less. But it’s a little unsatisfying. It doesn’t say much about what it implies—what a monad *means*. The problem is that it’s a *self-contained* definition. Even if you’re a beginning programmer, you have by now obtained a vast amount of knowledge related to programming, and this definition integrates with none of that.

In order to really *understand* what’s going on with monads, try to think about monads in terms of things you already know and connect them to a wider context. To develop some intuition for what monads *mean*, let’s look at another couple of monads and compare their behavior.

11.5.1 The identity monad

To distill monads to their essentials, let’s look at the simplest interesting specimen, the identity monad, given by the following type:

```
case class Id[A](value: A)
```



EXERCISE 11.17

Implement map and flatMap as methods on this class, and give an implementation for Monad[Id].